

Ejercicio 12

Necesitamos demostrar que toda expresión tiene un literal más que su cantidad de operadores. Los literales son las constantes y los rangos. Para esto se dispone de las siguientes definiciones:

data Nat = Z | S Nat

suma :: Nat -> Nat -> Nat

suma Z m = m — {S1}

suma (S n) m = S (suma n m) — {S2}

cantLit :: Expr -> Nat

cantLit (Const _) = S Z — {L1}

cantLit (Rango _ _) = S Z — {L2}

cantLit (Suma a b) = suma (cantLit a) (cantLit b) — {L3}

cantLit (Resta a b) = suma (cantLit a) (cantLit b) — {L4}

cantLit (Mult a b) = suma (cantLit a) (cantLit b) — {L5}

cantLit (Div a b) = suma (cantLit a) (cantLit b) — {L6}

cantOp :: Expr -> Nat

cantOp (Const _) = Z — {O1}

cantOp (Rango _ _) = Z — {O2}

cantOp (Suma a b) = S (suma (cantOp a) (cantOp b)) — {O3}

cantOp (Resta a b) = S (suma (cantOp a) (cantOp b)) — {O4}

cantOp (Mult a b) = S (suma (cantOp a) (cantOp b)) — {O5}

cantOp (Div a b) = S (suma (cantOp a) (cantOp b)) — {O6}

La propiedad a demostrar queda expresada de la siguiente manera:

$$\forall e :: \text{Expr} \quad \text{cantLit}(e) = S(\text{cantOp}(e))$$

a) Predicado unario

Queremos ver que la propiedad a demostrar es válida para cualquier expresión e del tipo de dato **Expr**. Para demostrar de mediante inducción estructural, es necesario definir un predicado unario que encapsule dicha propiedad. Este predicado es necesario porque la inducción estructural es un principio que permite demostrar que una propiedad P se cumple para todos los elementos de un tipo de dato recursivo. Este se basa en:

1. Demostrar que $P(e)$ se cumple para los casos base (constructores no recursivos).
2. Demostrar que si P se cumple para las subestructura, también se cumple para la estructura completa. El predicado $P(e)$ formaliza la proposición que debemos verificar en cada uno de estos pasos, permitiéndonos aplicar rigurosamente el esquema de inducción estructural sobre e . El predicado unario es el siguiente:

$$P(e) \equiv \text{cantLit}(e) = S(\text{cantOp}(e))$$

b) Esquema formal de inducción estructural

Dado el tipo de dato **Expr**, sabemos que este cuenta con 6 constructores (definidos en la primera parte del *TP*). Por lo que al definir su esquema formal, debemos separar en los constructores entre aquellos casos base y aquellos que se construyen de forma inductiva.

Casos base:

$$\begin{aligned} \forall a :: \text{Float}. P(\text{Const } a) \\ \forall a, b :: \text{Float}. P(\text{Rango } a \ b) \end{aligned}$$

Casos inductivos:

$$\begin{aligned} \forall a, b :: \text{Expr} \ (P(a) \wedge P(b)) &\Rightarrow P(\text{Suma } a \ b) \\ \forall a, b :: \text{Expr} \ (P(a) \wedge P(b)) &\Rightarrow P(\text{Resta } a \ b) \\ \forall a, b :: \text{Expr} \ (P(a) \wedge P(b)) &\Rightarrow P(\text{Mult } a \ b) \\ \forall a, b :: \text{Expr} \ (P(a) \wedge P(b)) &\Rightarrow P(\text{Div } a \ b) \end{aligned}$$

Donde $P(_)$ es el predicado unario definido en el inciso 'a' con su constructor específico para cada caso.

Si logramos probar tanto los casos base e inductivos, podemos concluir que se cumple la propiedad para toda la expresión.

c) Demostración de los casos

Caso base 1: Const a

$$\begin{aligned} \text{(Lado Izq)} : \quad & \text{cantLit}(\text{Const } a) &= S \ Z & \{L1\} \\ \text{(Lado Der)} : \quad & S(\text{cantOp}(\text{Const } a)) &= S \ Z & \{O1\} \end{aligned}$$

Como en ambos lados de la igualdad llego a lo mismo, queda demostrado que:

$$P(\text{Const } a) \equiv \text{cantLit}(\text{Const } a) = S(\text{cantOp}(\text{Const } a)).$$

Caso base 2: Rango a b

$$\begin{aligned} \text{(Lado Izq)} : \quad & \text{cantLit}(\text{Rango } a \ b) &= S \ Z & \{L2\} \\ \text{(Lado Der)} : \quad & S(\text{cantOp}(\text{Rango } a \ b)) &= S \ Z & \{O2\} \end{aligned}$$

Como en ambos lados de la igualdad llego a lo mismo, queda demostrado que:

$$P(\text{Rango } a \ b) \equiv \text{cantLit}(\text{Rango } a \ b) = S(\text{cantOp}(\text{Rango } a \ b)).$$

Caso inductivo: Suma a b

Asumo que valen las siguientes Hipótesis inductivas {HI}:

$$P(a) \equiv \text{cantLit}(a) = S(\text{cantOp}(a)), \quad P(b) \equiv \text{cantLit}(b) = S(\text{cantOp}(b)).$$

Queremos probar que:

$$P(\text{Suma } a \ b) \equiv \text{cantLit}(\text{Suma } a \ b) = S(\text{cantOp}(\text{Suma } a \ b)).$$

Lado Izquierdo:

$$\begin{aligned} \text{cantLit}(\text{Suma } a \ b) &= \text{suma}(\text{cantLit } a)(\text{cantLit } b) && \{L3\} \\ &= \text{suma}(S(\text{cantOp } a))(S(\text{cantOp } b)) && \text{Por H.I. en } a \text{ y } b \\ &= S(\text{suma}(\text{cantOp } a)(S(\text{cantOp } b))) && \{S2\} \\ &= S(\text{suma}(S(\text{cantOp } b))(\text{cantOp } a)) && \{\text{CONMUT}\} \\ &= S(S(\text{suma}(\text{cantOp } b)(\text{cantOp } a))) && \{S2\} \\ &= S(S(\text{suma}(\text{cantOp } a)(\text{cantOp } b))) && \{\text{CONMUT}\} \end{aligned}$$

Lado Derecho:

$$S(\text{cantOp}(\text{Suma } a\ b)) = S(S(\text{suma}(\text{cantOp } a)(\text{cantOp } b))) \quad \{O3\}$$

Conclusión: Hemos llegado a la misma expresión desde ambos lados de la propiedad., tanto en los casos base como en los inductivos. Por lo tanto, queda demostrado que para la expresion **suma a b** vale:

$$P(\text{Suma } a\ b) \equiv \text{cantLit}(\text{Suma } a\ b) = S(\text{cantOp}(\text{Suma } a\ b)).$$

□

(La demostración del resto de casos recursivos son análogos a este, por lo que siguen la misma estructura de la demostración de *Suma*).